

Types of Constructors in C# with Examples

In this article, I am going to discuss [Types of Constructors in C#](#) with Examples. Please read our previous article where we discussed the basic concepts of **Constructors in C#** with Examples.

Types of Constructors in C#

There are five types of constructors available in C#, they are as follows

1. **Default or Parameter Less Constructor**
2. **Parameterized Constructor**
3. **Copy Constructor**
4. **Static Constructor**
5. **Private Constructor**

Let's discuss each of these constructors in detail with examples.

Default or Parameterless Constructors in C#

If a constructor method does not take any parameters, then we call that a Default or Parameter Less Constructor. These constructors can be defined by a programmer explicitly or else will be defined implicitly provided there is no explicit constructor under the class. So, the Default or Parameterless Constructors are again classified into two types. They are as follows:

1. **System-Defined Default Constructor**
2. **User-Defined Default Constructor**

What is System Defined Default or Parameterless Constructor in C#?

As a programmer, if we are not defined any constructor explicitly in our program, then by default the compiler will provide one constructor at the time of compilation. That constructor is called a default constructor and the default constructor is parameterless. The default constructor will assign default values to the data members (non-static variables). As this constructor is created by the system this is also called a system-defined default constructor.

Example to understand System-Defined Implicit or Default Constructor in C#:

In the below example, within the Employee class, we have created a few variables. And then from the Main method, we created an instance of the Employee class and then printed the values of the variables.

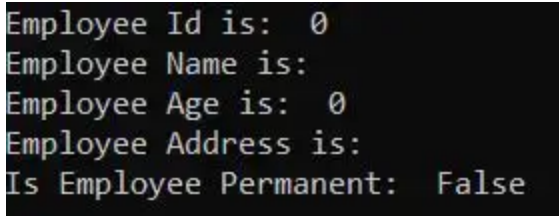
```
using System;
namespace ConstructorDemo
{
    class Employee
    {
        public int Id, Age;
        public string Address, Name;
    }
}
```

```

        public bool IsPermanent;
    }
    class Test
    {
        static void Main(string[] args)
        {
            Employee e1 = new Employee();
            Console.WriteLine("Employee Id is: " + e1.Id);
            Console.WriteLine("Employee Name is: " + e1.Name);
            Console.WriteLine("Employee Age is: " + e1.Age);
            Console.WriteLine("Employee Address is: " + e1.Address);
            Console.WriteLine("Is Employee Permanent: " + e1.IsPermanent);
            Console.ReadKey();
        }
    }
}

```

So, when you run the above code, you will see that default values based on the variable type are being printed on the console as shown in the below image. For numeric (in our example Id and Age), the default is 0. For string (in our example Address and Name) or object, the default value will be null (in this case nothing will be printed on the console) and for Boolean (in our example IsPermanent), the default value is false.

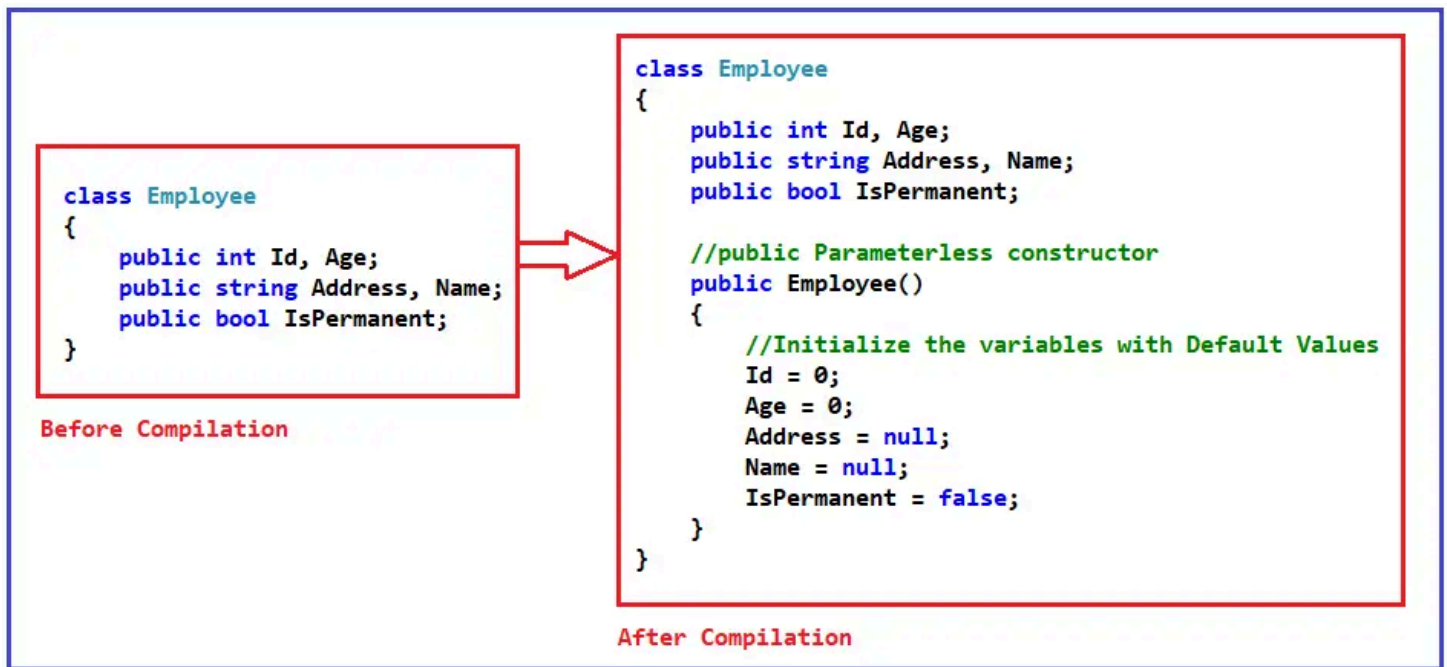


```

Employee Id is: 0
Employee Name is:
Employee Age is: 0
Employee Address is:
Is Employee Permanent: False

```

In our example, we have not specified these default values. Then who provided these default values and when? These default values are provided by the default constructor based on the variable data type and the compiler will provide the default constructor at the time of compilation. So, the Employee class will be converted as follows after compilation.



As you can see the Implicit Constructor which is always public and parameterless initialize the variables with default values and these default values you are seeing in the output. As this Implicit Constructor initializes the variables with default values, we also called this a System Defined Default Constructor.

Note: The point that you need to keep in mind is that the Compiler will only provide the default constructor if as a programmer we are not defined any constructor explicitly.

When do we need to provide the Constructor Explicitly?

If we want to execute some custom logic at the time of object creation, that logic may be object initialization logic or some other useful logic, then as a developer, we must provide the constructor explicitly in C#.

What is a User-Defined Default Constructor in C#?

The constructor which is defined by the user without any parameter is called the user-defined default constructor. This constructor does not accept any argument but as part of the constructor body, you can write your own logic.

Example to understand User-defined Default Constructor in C#

In the below example, within the Employee class, we have created a public parameterless constructor which is used to initialize the variables with some default hard-coded values. And then from the Main method, we created an instance of the Employee class and invoke the Display method.

```
using System;
namespace ConstructorDemo
{
    class Employee
    {
        public int Id, Age;
```

```

public string Address, Name;
public bool IsPermanent;
//User Defined Default Constructor
public Employee()
{
    Id = 100;
    Age = 30;
    Address = "Bhubaneswar";
    Name = "Anurag";
    IsPermanent = true;
}
public void Display()
{
    Console.WriteLine("Employee Id is: " + Id);
    Console.WriteLine("Employee Age is: " + Age);
    Console.WriteLine("Employee Address is: " + Address);
    Console.WriteLine("Employee Name is: " + Name);
    Console.WriteLine("Is Employee Permanent: " + IsPermanent);
}
}
class Program
{
    static void Main(string[] args)
    {
        Employee e1 = new Employee();
        e1.Display();
        Console.ReadKey();
    }
}

```

Output:

```

Employee Id is: 100
Employee Age is: 30
Employee Address is: Bhubaneswar
Employee Name is: Anurag
Is Employee Permanent: True

```

The Employee class constructor is also called a Default Constructor because it is public and parameter-less.

```

class Employee
{
    public int Id, Age;
    public string Address, Name;
    public bool IsPermanent;

    //User Defined Default Constructor
    public Employee()
    {
        Id = 100;
        Age = 30;
        Address = "Bhubaneswar";
        Name = "Anurag";
        IsPermanent = true;
    }

    public void Display()...
}

```

And more importantly, it does not matter how many objects are being created for the Employee class, each instance or object is initialized with the same set of values. This is the reason we call it a Default Constructor. As this constructor is created by the user, so we call it a User-Defined Default Constructor.

The drawback of the above user-defined default constructor is that each and every instance (i.e. object) of the class will be initialized (assigned) with the same set of values. That means it is not possible to initialize each instance of the class with different values. For a better understanding, please modify the Main method as follows and see the output.

```

class Program
{
    static void Main(string[] args)
    {
        Employee e1 = new Employee();
        e1.Display();
        Employee e2 = new Employee();
        Console.WriteLine();
        e2.Display();
        Console.ReadKey();
    }
}

```

Output:

```
Employee Id is: 100
Employee Age is: 30
Employee Address is: Bhubaneswar
Employee Name is: Anurag
Is Employee Permanent: True

Employee Id is: 100
Employee Age is: 30
Employee Address is: Bhubaneswar
Employee Name is: Anurag
Is Employee Permanent: True
```

When should we define a parameterized constructor in a class?

If we want to initialize the object dynamically with the user-given values or if we want to initialize each instance of a class with a different set of values then we need to use the Parameterized Constructor in C#. The advantage is that we can initialize each instance with different values.

What is Parameterized Constructor in C#?

If a constructor method is defined with parameters, we call it a Parameterized Constructor in C#, and these constructors are defined by the programmers only but never can be defined implicitly. So, in simple words, we can say that the developer-given constructor with parameters is called Parameterized Constructor in C#.

Let us understand Parameterized Constructor in C# with Examples. Please have a look at the below code. Here, we have a class called ParameterizedConstructor, and this class has one constructor which is taking one integer parameter. As this constructor takes a parameter, we call it a Parameterized Constructor. And in this constructor, we are printing the i value.

```
public class ParameterizedConstructor
{
    //Constructor with Parameter is called Parameterized Constructor
    public ParameterizedConstructor(int i)
    {
        Console.WriteLine($"Parameterized Constructor is Called : {i}");
    }
}
```

And now, when you are going to create an instance of the ParameterizedConstructor class, it will ask you for the integer parameter value as shown in the below image.

```
static void Main(string[] args)
```

```
{  
    ParameterizedConstructor obj = new ParameterizedConstructor()  
    Console.ReadKey();  
}
```

ParameterizedConstructor(int i)



It asking to supply the value for int i

Now, you can pass an integer value. Suppose, we pass the value 10, then that value 10 will directly come to variable i which is then printed on the console. For a better understanding, please have a look at the below image.

```
static void Main(string[] args)
```

```
{  
    ParameterizedConstructor obj = new ParameterizedConstructor(10);  
    Console.ReadKey();  
}
```

```
public class ParameterizedConstructor
```

```
{  
    public ParameterizedConstructor(int i)  
    {  
        Console.WriteLine($"Parameterized Constructor is Called : {i}");  
    }  
}
```

The complete example code is given below.

```
using System;  
namespace ConstructorDemo  
{  
    class Program  
    {  
        static void Main(string[] args)  
        {  
            ParameterizedConstructor obj = new ParameterizedConstructor(10);  
            Console.ReadKey();  
        }  
    }  
    public class ParameterizedConstructor
```

```

{
    public ParameterizedConstructor(int i)
    {
        Console.WriteLine($"Parameterized Constructor is Called: {i}");
    }
}

```

Output: Parameterized Constructor is Called: 10

So, in this way, we can create any number of instances of the class, and while creating the instance we can pass different values and those values will go and sit in the variable i. In the below example, we are creating two different instances of ParameterizedConstructor class with two different values.

```

using System;
namespace ConstructorDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            ParameterizedConstructor obj1 = new ParameterizedConstructor(10);
            ParameterizedConstructor obj2 = new ParameterizedConstructor(20);
            Console.ReadKey();
        }
    }
    public class ParameterizedConstructor
    {
        public ParameterizedConstructor(int i)
        {
            Console.WriteLine($"Parameterized Constructor is Called : {i}");
        }
    }
}

```

Output:

```

Parameterized Constructor is Called : 10
Parameterized Constructor is Called : 20

```

When should we use Parameterized Constructor in C#?

With the help of a Parameterized constructor, we can initialize each instance of the class with a different set of values. That means using parameterized constructor we can store a different set of values in different instances created in the class.

Let us understand this with an example. Please have a look at the below class. This is the same class that we worked on in our previous example with some changes. Now, in the class, I have declared a variable called x. Remember, this variable x is initialized with default value only i.e. 0. Why default value? Because we didn't assign a value and when we don't assign a value, the constructor will take the responsibility of

assigning a value. And the value for x is going to be 0. Then we created a method called Display where we print the x value.

```
public class ParameterizedConstructor
{
    int x;
    public ParameterizedConstructor(int i)
    {
        Console.WriteLine($"Parameterized Constructor is Called : {i}");
    }
    public void Display()
    {
        Console.WriteLine($"Value of X = {x}");
    }
}
```

Now, let us call this Display method using two different instances. The complete example code is given below.

```
using System;
namespace ConstructorDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            ParameterizedConstructor obj1 = new ParameterizedConstructor(10);
            obj1.Display();
            ParameterizedConstructor obj2 = new ParameterizedConstructor(20);
            obj2.Display();
            Console.ReadKey();
        }
    }
    public class ParameterizedConstructor
    {
        int x;
        public ParameterizedConstructor(int i)
        {
            Console.WriteLine($"Parameterized Constructor is Called : {i}");
        }
        public void Display()
        {
            Console.WriteLine($"Value of X = {x}");
        }
    }
}
```

Output:

```
Parameterized Constructor is Called : 10  
Value of X = 0  
Parameterized Constructor is Called : 20  
Value of X = 0
```

As you can see in the above image, for both the instances it is printing the x value as 0. In instance one we pass 10 to the constructor and in instance two we pass 20 to the constructor. So, can we use the values 10 and 20 in place of x? Yes, it is possible. How it is possible? By using Parameterized constructor only. Let us see how we can do this. Inside the constructor, we can assign the x variable with the value of i as shown in the below image.

```
public class ParameterizedConstructor  
{  
    int x;  
    public ParameterizedConstructor(int i)  
    {  
        //Initializing the variable  
        x = i;  
        Console.WriteLine($"Parameterized Constructor is Called : {i}");  
    }  
    public void Display()...  
}
```

With the above changes in place, now if you run the application, then it will print 10 and 20 for the x variable. The complete example code is given below.

```
using System;  
namespace ConstructorDemo  
{  
    class Program  
    {  
        static void Main(string[] args)  
        {  
            ParameterizedConstructor obj1 = new ParameterizedConstructor(10);  
            obj1.Display();  
            ParameterizedConstructor obj2 = new ParameterizedConstructor(20);  
            obj2.Display();  
            Console.ReadKey();  
        }  
    }  
    public class ParameterizedConstructor  
    {  
        int x;  
        public ParameterizedConstructor(int i)  
        {
```

```

        //Initializing the variable
        x = i;
        Console.WriteLine($"Parameterized Constructor is Called : {i}");
    }
    public void Display()
    {
        Console.WriteLine($"Value of X = {x}");
    }
}
}

```

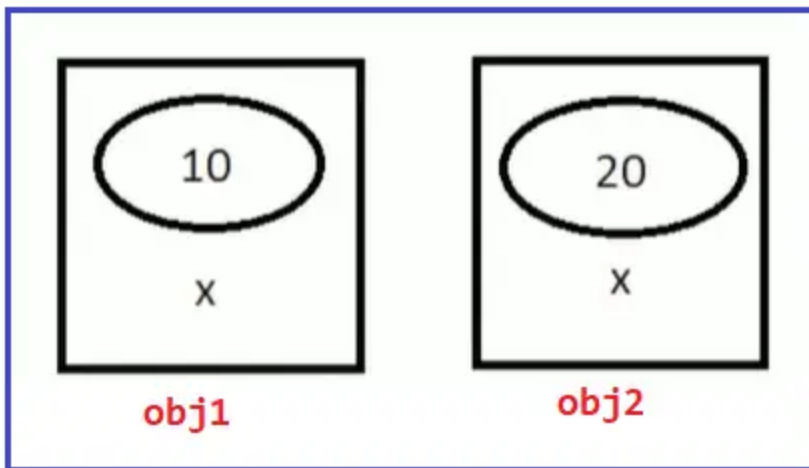
Output:

```

Parameterized Constructor is Called : 10
Value of X = 10
Parameterized Constructor is Called : 20
Value of X = 20

```

As we have created two instances separately i.e. obj1 and obj2. So, internally two copies of the x variable are available in the memory for us. For a better understanding, please have a look at the below diagram. As you can see in the below image, we have two separate objects i.e. obj1 and obj2. For obj1 we have one copy of the x variable with the value 10 is there and the obj2 another copy of the x variable with the value 20 is there in the memory.



Copy Constructor in C#:

If we want to create multiple instances with the same values then we need to use the copy constructor in C#, in a copy constructor the constructor takes the same class as a parameter to it.

Let us understand Copy Constructor in C# with Examples. Please have a look at the below code. This is the same code that we have written in our previous example.

```

using System;
namespace ConstructorDemo
{

```

```

class Program
{
    static void Main(string[] args)
    {
        CopyConstructor obj1 = new CopyConstructor(10);
        obj1.Display();
        Console.ReadKey();
    }
}
public class CopyConstructor
{
    int x;
    public CopyConstructor(int i)
    {
        x = i;
    }
    public void Display()
    {
        Console.WriteLine($"Value of X = {x}");
    }
}
}

```

Output: Value of X = 10

Now, I want to create another instance with the same value. Then I can create another instance as follows.

```

static void Main(string[] args)
{
    CopyConstructor obj1 = new CopyConstructor(10);
    obj1.Display();
    CopyConstructor obj2 = new CopyConstructor(10);
    obj2.Display();
    Console.ReadKey();
}

```

See, passing one parameter is not a difficult task. Suppose, the constructor takes 10 or 20 parameters, then it is a time-consuming and error-prone process to pass the same 10 or 20 parameters. We can overcome this problem by using Copy Constructor in C#. The copy constructor takes a parameter of the same class type. How we can pass a class name as a parameter. This is because a class is a user-defined data type. For a better understanding, please have a look at the below image.

```

public class CopyConstructor
{
    int x;

    //Parameterized Constructor
    public CopyConstructor(int i)
    {
        x = i;
    }

    //Copy Constructor
    public CopyConstructor(CopyConstructor obj)
    {
        x = obj.x;
    }

    public void Display()...
}

```

With the above changes, now you can see we have two constructors as shown in the below image. One constructor takes an int as a parameter and the other constructor takes the CopyConstructor type as a parameter.

```

static void Main(string[] args)
{
    CopyConstructor obj1 = new CopyConstructor(10);
    obj1.Display();
    CopyConstructor obj2 = new CopyConstructor(i);
    obj2.Display();
    Console.ReadKey();
}

```

▲ 1 of 2 ▼ CopyConstructor(CopyConstructor obj)

Now, the class have two constructors

So, can we define multiple constructors in a class? Yes, we can. Constructors can be overloaded in C#. The complete example code is given below. Now, we are passing obj1 as a parameter to the copy constructor.

```

using System;
namespace ConstructorDemo
{
    class Program
    {

```

```

static void Main(string[] args)
{
    CopyConstructor obj1 = new CopyConstructor(10);
    obj1.Display();
    CopyConstructor obj2 = new CopyConstructor(obj1);
    obj2.Display();
    Console.ReadKey();
}
}
public class CopyConstructor
{
    int x;
    //Parameterized Constructor
    public CopyConstructor(int i)
    {
        x = i;
    }
    //Copy Constructor
    public CopyConstructor(CopyConstructor obj)
    {
        x = obj.x;
    }
    public void Display()
    {
        Console.WriteLine($"Value of X = {x}");
    }
}
}

```

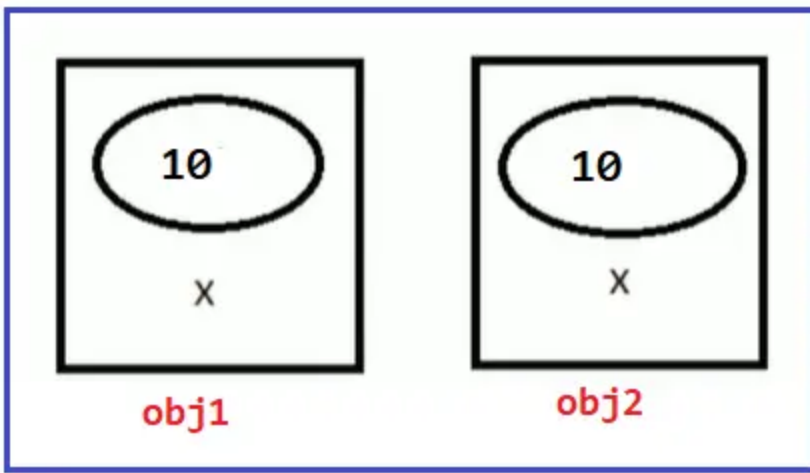
Now, if you run the application then you see that both the instances will have the same x value as shown in the below image,

```

Value of X = 10
Value of X = 10

```

Now, the memory is separate for each instance but the value is going to be the same for both instances. For a better understanding, please have a look at the below diagram.



How many Constructors can be Defined in a Class in C#?

In C#, within a class, we can define any number of constructors. But the most important point that you need to remember is that each and every constructor must have a different signature. A different signature means the number, type, and parameter order should be different. So, in a class, we can define one no-argument constructor plus 'n' number of parameterized constructors in C#.

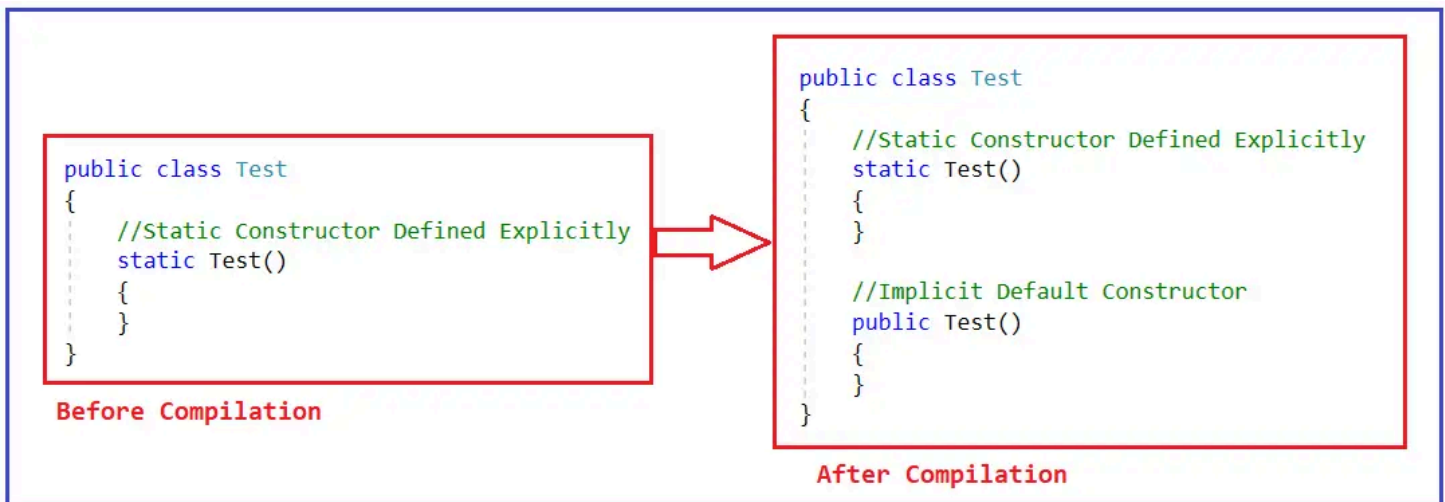
Static Constructor in C#

In C#, it is also possible to create a constructor as static and when we do so, it is called a Static Constructor. If a constructor is declared explicitly by using the static modifier, then it is called a static constructor in C#. All the constructors we defined till now are non-static or instance constructors.

For a better understanding, please have a look at the below example. In a static constructor, you cannot use any access specifiers like public, private, and protected.

```
public class Test
{
    //Static Constructor
    static Test()
    {
    }
}
```

But, when you will compile the above, the compiler will provide the default parameter less constructor. For a better understanding, please have a look at the below code.



Points to Remember while working with Static Constructors in C#:

Point1:

If a class contains any static variables, then only implicit static constructors come into the picture otherwise we must be defined them explicitly. On the other hand, non-static constructors will be implicitly defined in every class (except the static class) provided we didn't define any constructor explicitly.

Point2:

Static Constructors are responsible for initializing static variables and these constructors are never called explicitly. They are called Implicitly and moreover, these constructors are the first to execute in any class. For a better understanding, please have a look at the below example. Here, we have defined one static constructor, and please observe from the Main method we are not calling the Static constructor.

```
using System;
namespace ConstructorDemo
{
    public class StaticConstructor
    {
        static StaticConstructor()
        {
            Console.WriteLine("Static Constructor Executed!");
        }
        static void Main(string[] args)
        {
            Console.WriteLine("Main Method Exceution Started...");
            Console.ReadKey();
        }
    }
}
```

Now, when you execute the above code, the Static constructor will execute first and then the main method. And this proves the following output.

```
Static Constructor Executed!  
Main Method Execution Started...
```

The non-static constructors are never called implicitly, they are always called explicitly whereas the static constructor is never called explicitly, they are always going to be called implicitly. How does the execution happen? See, the Main method is the starting point of execution, and in this case no difference. The program execution will start from the Main method but before executing any statement inside the Main method, it will first execute the Static constructor and once the Static Constructor execution is completed, then it will continue the execution of the Main method. So, the static constructor is the first block of code in a class to be executed.

Point3:

Static Constructors cannot be parameterized, so overloading of the static constructors is not possible in C#. Now, the question is why we cannot parameterize the static constructor? The answer is simple. The static constructors are executed implicitly and hence we never get a chance to pass a value. And as the static constructor is the first block to be executed in a class, and hence there is no chance to pass a value.

Points To Remember About Static Constructor in C#:

1. There can be only one static constructor in a class.
2. It can't be called explicitly, it is always called implicitly.
3. The static constructor should be without any parameters.
4. It can only access the static members of the class.
5. There should not be any access specifiers in the static constructor definition.
6. If a class is static then we cannot create the object for the static class.
7. It is called automatically to initialize the static members.
8. Static constructor will be invoked only once i.e. at the time of class loading.

Can we initialize non-static data members within a static constructor in C#?

It is not possible to initialize non-static data members within a static constructor, it raises a compilation error. For a better understanding, please have a look at the following example.

```

class Example
{
    int i;
    static int j;

    static Example()
    {
        j = 100; //Allowed
        i = 101; //Not Allowed
    }
}

```

 (field) `int Example.i`

An object reference is required for the non-static field, method, or property 'Example.i'

Can we initialize static data fields within a non-static constructor in C#?

Yes, we can initialize static data members within a non-static constructor. Consider the following example for better understanding:

```

class Example
{
    int i;
    static int j;

    public Example()
    {
        i = 100;
        j = 100; //Allowed
    }
}

```

What is a Private Constructor in C#?

In C#, it is also possible to create a constructor as private. The constructor whose accessibility is private is known as a private constructor. When a class contains a private constructor then we cannot create an object for the class outside of the class. So, private constructors are used to create an object for the class within the same class. Generally, private constructors are used in the Remoting concept.

Example to understand Private Constructor in C#

```

using System;

```

```
namespace ConstructorDemo
{
    class Program
    {
        private Program()
        {
            Console.WriteLine("This is private constructor");
        }
        static void Main(string[] args)
        {
            Program p = new Program();
            Console.WriteLine("Main method");
            Console.ReadKey();
        }
    }
}
```

Points To Remember about C# Private Constructor:

1. Using Private Constructor in C# we can implement the singleton design pattern.
2. We need to use the private constructor in C# when the class contains only static members.
3. Using a private constructor is not possible to create an instance from outside the class.

In the next article, I am going to discuss [Why Do We Need Constructors in C#](#) with Examples. Here, in this article, I try to explain the **Types of Constructors in C#** with Examples. I hope you enjoy this Types of Constructors in C# article. Please give your feedback, suggestions, and questions about this article in the comment section.